

1 Assignment Overview

What you will implement

1. Byte-pair encoding (BPE) tokenizer
2. Transformer language model (LM)
3. The cross-entropy loss function and the AdamW optimizer
4. The training loop, with support for serializing and loading model and optimizer state

DONE

What you will run

1. Train a BPE tokenizer on the TinyStories dataset.
2. Run your trained tokenizer on the dataset to convert it into a sequence of integer IDs.
3. Train a Transformer LM on the TinyStories dataset.
4. Generate samples and evaluate perplexity using the trained Transformer LM.
5. Train models on OpenWebText and submit your attained perplexities to a leaderboard.

DONE

2 Byte-Pair Encoding (BPE) Tokenizer

2.1 The Unicode Standard

In Python, you can use the `ord()` function to convert a single Unicode character into its integer representation. The `chr()` function converts an integer Unicode code point into a string with the corresponding character.

Problem (unicode1): Understanding Unicode (1 point)

- (a) What Unicode character does `chr(0)` return?

Answer: `'\x00'`

- (b) How does this character's string representation `__repr__()` differ from its printed representation?

Answer: The `__repr__()` is designed to be **unambiguous** and is mainly for developers, showing the technical details of the object. On the other hand, the printed representation is meant to be **readable** and user-friendly.

- (c) What happens when this character occurs in text? It may be helpful to play around with the following in your Python interpreter and see if it matches your expectations:

Answer: Code below.

```
>>> chr(0)
'\x00'
>>> print(chr(0))

>>> "this is a test" + chr(0) + "string"
'this is a test\x00string'
>>> print("this is a test" + chr(0) + "string")
this is a teststring
```

2.2 Unicode Encodings

- UTF-8: The Efficient Variable-Length Encoder, It uses 1 to 4 bytes per character.
- UTF-16: The Middle Ground, It uses either 2 or 4 bytes per character.
- UTF-32: The Simple Fixed-Length Encoder, It uses exactly 4 bytes for every character, regardless of what it is.

Problem (unicode2): Unicode Encodings (3 point)

- (a) What are some reasons to prefer training our tokenizer on UTF-8 encoded bytes, rather than UTF-16 or UTF-32? It may be helpful to compare the output of these encodings for various input strings.

Answer: `'\x00'`

- (b) Consider the following (incorrect) function, which is intended to decode a UTF-8 byte string into a Unicode string. Why is this function incorrect? Provide an example of an input byte string that yields incorrect results.

```
def decode_utf8_bytes_to_str_wrong(bytestring: bytes):  
    return "".join([bytes([b]).decode("utf-8") for b in bytestring])  
  
>>> decode_utf8_bytes_to_str_wrong("hello".encode("utf-8"))  
'hello'
```

Example: `b'\xe4\xb8\xad'`.

Answer: This function is incorrect because it attempts to decode each byte individually, which fails for any character encoded as a multi-byte sequence, resulting in a `UnicodeDecodeError`.

- (c) Give a two byte sequence that does not decode to any Unicode character(s).

Example: `b'\xff\xff'`.

Answer: This sequence is invalid because the byte `0xFF` is explicitly forbidden in the UTF-8 standard and does not correspond to any valid lead or continuation byte in the encoding scheme.

2.3 Subword Tokenization

Sennrich et al. [2016] propose to use byte-pair encoding (BPE; Gage, 1994), a compression algorithm that iteratively replaces (“merges”) the most frequent pair of bytes with a single, new unused index.

2.4 BPE Tokenizer Training

Vocabulary initialization

Pre-tokenization Use regular expressions to split the sentence into independent punctuation marks and independent items that may include spaces, and improve the merging effect by counting the occurrence frequency of these independent items.

```
>>> PAT = r"'(?:[sdmt]|ll|ve|re)| ?\p{L}+| ?\p{N}+| ?[\s\p{L}\p{N}]+\s+(?!S)|\s+"
>>> # requires `regex` package
>>> import regex as re
>>> re.findall(PAT, "some text that i'll pre-tokenize")
['some', ' text', ' that', ' i', "'ll", ' pre', '-', 'tokenize']
```

Compute BPE merges During the BPE merging process, two **principles** must be followed:

- Do not **cross** pre-tokenization boundaries (this is already guaranteed algorithmically);
- Handle issues with a **reproducible conflict resolution strategy**, such as selecting the lexicographically largest word pair to merge when the occurrence counts are the same.

Special tokens

2.5 Experimenting with BPE Tokenizer Training

Parallelizing pre-tokenization Using the provided segmentation code and the `multiprocessing` library, parallelize the tokenization statistics.

Removing special tokens before pre-tokenization

Optimizing the merging step After performing a statistical pass to obtain a tokenization table with frequencies, perform merging only within that tokenization table.

Low-Resource/Downscaling Tip: Profiling

You should use profiling tools like `cProfile` or `scalene` to identify the bottlenecks in your implementation, and focus on optimizing those.

Problem (train_bpe): BPE Tokenizer Training (15 points)

Deliverable: Write a function that, given a path to an input text file, trains a (byte-level) BPE tokenizer. Your BPE training function should handle (at least) the following input parameters:

input_path: `str` Path to a text file with BPE tokenizer training data.

vocab_size: `int` A positive integer that defines the maximum final vocabulary size (including the initial byte vocabulary, vocabulary items produced from merging, and any special tokens).

special_tokens: `list[str]` A list of strings to add to the vocabulary. These special tokens do not otherwise affect BPE training.

Your BPE training function should return the resulting vocabulary and merges:

vocab: `dict[int, bytes]` The tokenizer vocabulary, a mapping from `int` (token ID in the vocabulary) to `bytes` (token bytes).

merges: `list[tuple[bytes, bytes]]` A list of BPE merges produced from training. Each list item is a `tuple` of `bytes` (`<token1>`, `<token2>`), representing that `<token1>` was merged with `<token2>`. The merges should be ordered by order of creation.

Insights: BPE Tokenizer Training

Python plus Rust: Python is used for rapid prototyping, while Rust handles the actual deployment of the product.

Glitch Tokens: Due to insufficiently strict data cleaning, some meaningless words—namely **Mojibake**—may appear during the BPE training process. Moreover, because of BPE’s **greedy, unsupervised, and non-semantic nature**, a large number of such meaningless word merges occur, turning them into abnormally long and nonsensical words.

Vocabulary Bloat (Dead Tokens): Because BPE is incremental and never removes merges, the intermediate products of long words permanently occupy the vocabulary, leading to wasted model parameters and tokenization fragmentation. In industry, there are three approaches to address this issue: First, absolutely aggressive data cleaning; second, stricter regex filtering. Both of these are common methods for limiting glitch tokens. The third approach is to switch to the **Unigram** algorithm, which is frequently used by Google-family models (such as T5 and some Llama variants).

Unigram: Unigram works by "subtraction": it first initializes an extremely large vocabulary (e.g., 100,000 words) and **then calculates how much the overall corpus compression rate would be affected if a given word were removed**. Those useless intermediate composite words are directly eliminated and discarded during training.

Problem (train_bpe_tinystories): BPE Training on TinyStories (2 points)

- (a) Train a byte-level BPE tokenizer on the TinyStories dataset, using a maximum vocabulary size of 10,000. Make sure to add the TinyStories `<|endoftext|>` special token to the vocabulary. Serialize the resulting vocabulary and merges to disk for further inspection. How many hours and memory did training take? What is the longest token in the vocabulary? Does it make sense?

Resource requirements: ≤ 30 minutes (no GPUs), ≤ 30 GB RAM

Resource used: 10.78 seconds, 2.6GB RAM

Longest token: 'disappointment'

- (b) Profile your code. What part of the tokenizer training process takes the most time?

Answer: `process_chunk_with_special_tokens` takes up the vast majority of the time.

Problem (train_bpe_expts_owt): BPE Training on OpenWebText (2 points)

- (a) Train a byte-level BPE tokenizer on the OpenWebText dataset, using a maximum vocabulary size of 32,000. Serialize the resulting vocabulary and merges to disk for further inspection. What is the longest token in the vocabulary? Does it make sense?

Resource requirements: ≤ 12 hours (no GPUs), ≤ 100 GB RAM

Resource used: 6 minutes, 7.0GB RAM

Longest token: '_____'

- (b) Compare and contrast the tokenizer that you get training on TinyStories versus OpenWebText.

Answer: Because TinyStories has an extremely limited vocabulary, a 10k vocab size forces the BPE tokenizer to over-merge, creating tokens that represent entire multi-word phrases. In contrast, OpenWebText's diverse internet text requires the BPE algorithm to rely heavily on subwords (like prefixes and suffixes) to represent its massive vocabulary within a 32k limit, resulting in shorter, more fragmented tokens for rare words.

2.6 BPE Tokenizer: Encoding and Decoding

2.6.1 Encoding text

Special tokens.

Memory considerations.

2.6.2 Decoding text

Problem (tokenizer): Implementing the tokenizer (15 points)

Deliverable: Implement a `Tokenizer` class that, given a vocabulary and a list of merges, encodes text into integer IDs and decodes integer IDs into text. Your tokenizer should also support user-provided special tokens (appending them to the vocabulary if they aren't already there).

Answer: code in appendix.

2.7 Experiments

Problem (tokenizer_experiments): Experiments with tokenizers (4 points)

- (a) Sample 10 documents from TinyStories and OpenWebText. Using your previously-trained TinyStories and OpenWebText tokenizers (10K and 32K vocabulary size, respectively), encode these sampled documents into integer IDs. What is each tokenizer's compression ratio (bytes/token)?

Answer: The TinyStories tokenizer achieves a higher compression ratio (5.5 bytes/token) because its vocabulary easily memorizes long, frequent whole words from its restricted domain. In contrast, the OpenWebText tokenizer has a lower compression ratio (3.8 bytes/token) as it must rely heavily on shorter subwords to encode a highly diverse vocabulary.

- (b) What happens if you tokenize your OpenWebText sample with the TinyStories tokenizer? Compare the compression ratio and/or qualitatively describe what happens.

Answer: Tokenizing OpenWebText with the TinyStories tokenizer causes a severe domain mismatch, forcing the tokenizer to aggressively fragment unfamiliar web text into individual characters or very small byte sequences. Consequently, the token count explodes, resulting in a dramatically lower (worse) compression ratio compared to using the native OpenWebText tokenizer.

- (c) Estimate the throughput of your tokenizer (e.g., in bytes/second). How long would it take to tokenize the Pile dataset (825GB of text)?

Answer: The throughput of the tokenizer depends on the implementation details, but a rough estimate for a modern CPU might be around 100 MB/second. Given the Pile dataset's size of 825GB, it would take approximately 8.25 hours to tokenize the entire dataset.

- (d) Using your TinyStories and OpenWebText tokenizers, encode the respective training and development datasets into a sequence of integer token IDs. We'll use this later to train our language model. We recommend serializing the token IDs as a NumPy array of datatype `uint16`. Why is `uint16` an appropriate choice?

Answer: `uint16` is an appropriate choice because it can represent all possible token IDs within the range of 0 to 65,535, which is sufficient for most practical applications and allows for efficient memory usage.

3 Transformer Language Model Architecture

3.1 Transformer LM

3.1.1 Token Embeddings

3.1.2 Pre-norm Transformer Block

3.2 Output Normalization and Embedding

3.3 Remark: Batching, Einsum and Efficient Computation

3.3.1 Mathematical Notation and Memory Ordering

3.4 Basic Building Blocks: Linear and Embedding Modules

3.4.1 Parameter Initialization

3.4.2 Linear Module

Problem (linear): Implementing the linear module

(1 point)

Deliverable: Implement a `Linear` class that inherits from `torch.nn.Module` and performs a linear transformation. Your implementation should follow the interface of PyTorch's built-in `nn.Linear` module, except for not having a `bias` argument or parameter. We recommend the following interface:

`def __init__(self, in_features, out_features, device=None, dtype=None)` Construct a linear transformation module. This function should accept the following parameters:

`in_features: int` final dimension of the input

`out_features: int` final dimension of the output

`device: torch.device | None = None` Device to store the parameters on

`dtype: torch.dtype | None = None` Data type of the parameters

`def forward(self, x: torch.Tensor) -> torch.Tensor` Apply the linear transformation to the input.

Make sure to:

- subclass `nn.Module`
- call the superclass constructor
- construct and store your parameter as W (not W^T) for memory ordering reasons, putting it in an `nn.Parameter`
- of course, don't use `nn.Linear` or `nn.functional.linear`

Code below;

3.4.3 Embedding Module

Problem (embedding): Implement the embedding module**(1 point)**

Deliverable: Implement the `Embedding` class that inherits from `torch.nn.Module` and performs an embedding lookup. Your implementation should follow the interface of PyTorch's built-in `nn.Embedding` module. We recommend the following interface:

```
def __init__(self, num_embeddings, embedding_dim, device=None, dtype=None)
```

Construct an embedding module. This function should accept the following parameters:

`num_embeddings`: `int` Size of the vocabulary

`embedding_dim`: `int` Dimension of the embedding vectors, i.e., d_{model}

`device`: `torch.device` | `None` = `None` Device to store the parameters on

`dtype`: `torch.dtype` | `None` = `None` Data type of the parameters

```
def forward(self, token_ids: torch.Tensor) -> torch.Tensor
```

Lookup the embedding vectors for the given token IDs.

Make sure to:

- subclass `nn.Module`
- call the superclass constructor
- initialize your embedding matrix as a `nn.Parameter`
- store the embedding matrix with the `d_model` being the final dimension
- of course, don't use `nn.Embedding` or `nn.functional.embedding`

Again, use the settings from above for initialization, and use `torch.nn.init.trunc_normal_` to initialize the weights.

To test your implementation, implement the test adapter at `[adapters.run_embedding]`. Then, run `uv run pytest -k test_embedding`.

3.5 Pre-Norm Transformer Block

3.5.1 Root Mean Square Layer Normalization

Problem (rmsnorm): Root Mean Square Layer Normalization (1 point)

Deliverable: Implement RMSNorm as a `torch.nn.Module`. We recommend the following interface:

```
def __init__(self, d_model: int, eps: float = 1e-5, device=None, dtype=None)
```

Construct the RMSNorm module. This function should accept the following parameters:

`d_model: int` Hidden dimension of the model

`eps: float = 1e-5` Epsilon value for numerical stability

`device: torch.device | None = None` Device to store the parameters on

`dtype: torch.dtype | None = None` Data type of the parameters

```
def forward(self, x: torch.Tensor) -> torch.Tensor
```

Process an input tensor of shape `(batch_size, sequence_length, d_model)` and return a tensor of the same shape.

Note: Remember to upcast your input to `torch.float32` before performing the normalization (and later downcast to the original dtype), as described above.

To test your implementation, implement the test adapter at `[adapters.run_rmsnorm]`. Then, run `uv run pytest -k test_rmsnorm`.

3.5.2 Position-Wise Feed-Forward Network

Problem (positionwise_feedforward): Implement the position-wise feed-forward network (2 points)

Deliverable: Implement the SwiGLU feed-forward network, composed of a SiLU activation function and a GLU.

Note: in this particular case, you should feel free to use `torch.sigmoid` in your implementation for numerical stability.

You should set d_{ff} to approximately $\frac{8}{3} \times d_{\text{model}}$ in your implementation, while ensuring that the dimensionality of the inner feed-forward layer is a multiple of 64 to make good use of your hardware. To test your implementation against our provided tests, you will need to implement the test adapter at `[adapters.run_swiglu]`. Then, run `uv run pytest -k test_swiglu` to test your implementation.

3.5.3 Relative Positional Embeddings

Problem (rope): Implement RoPE (2 points)

Deliverable: Implement a class `RotaryPositionalEmbedding` that applies RoPE to the input tensor. The following interface is recommended:

```
def __init__(self, theta: float, d_k: int, max_seq_len: int, device=None)
```

Construct the RoPE module and create buffers if needed.

`theta: float` Θ value for the RoPE

`d_k: int` dimension of query and key vectors

`max_seq_len: int` Maximum sequence length that will be inputted

`device: torch.device | None = None` Device to store the buffer on

```
def forward(self, x: torch.Tensor, token_positions: torch.Tensor) -> torch.Tensor
```

Process an input tensor of shape `(..., seq_len, d_k)` and return a tensor of the same shape. Note that you should tolerate `x` with an arbitrary number of batch dimensions. You should assume that the token positions are a tensor of shape `(..., seq_len)` specifying the token positions of `x` along the sequence dimension.

You should use the token positions to slice your (possibly precomputed) `cos` and `sin` tensors along the sequence dimension.

To test your implementation, complete `[adapters.run_rope]` and make sure it passes `uv run pytest -k test_rope`.

3.5.4 Scaled Dot-Product Attention

Problem (softmax): Implement softmax

(1 point)

Deliverable: Write a function to apply the softmax operation on a tensor. Your function should take two parameters: a tensor and a *dimension* `i`, and apply softmax to the `i`-th dimension of the input tensor. The output tensor should have the same shape as the input tensor, but its `i`-th dimension will now have a normalized probability distribution. Use the trick of subtracting the maximum value in the `i`-th dimension from all elements of the `i`-th dimension to avoid numerical stability issues.

To test your implementation, complete `[adapters.run_softmax]` and make sure it passes `uv run pytest -k test_softmax_matches_pytorch`.

Problem (scaled_dot_product_attention): Implement scaled dot-product attention (5 points)

Deliverable: Implement the scaled dot-product attention function. Your implementation should handle keys and queries of shape `(batch_size, ..., seq_len, d_k)` and values of shape `(batch_size, ..., seq_len, d_v)`, where `...` represents any number of other batch-like dimensions (if provided). The implementation should return an output with the shape `(batch_size, ..., d_v)`. See section 3.3 for a discussion on batch-like dimensions.

Your implementation should also support an optional user-provided boolean mask of shape `(seq_len, seq_len)`. The attention probabilities of positions with a mask value of `True` should collectively sum to 1, and the attention probabilities of positions with a mask value of `False` should be zero.

To test your implementation against our provided tests, you will need to implement the test adapter at `[adapters.run_scaled_dot_product_attention]`.

`uv run pytest -k test_scaled_dot_product_attention` tests your implementation on third-order input tensors, while `uv run pytest -k test_4d_scaled_dot_product_attention` tests your implementation on fourth-order input tensors.

A Complete Source Code

A.1 Fast BPE Implementation (Rust)

```

1  use pyo3::prelude::*;
2  use std::collections::{HashMap, HashSet, BinaryHeap};
3  use std::cmp::Ordering;
4
5  #[derive(Eq, PartialEq)]
6  struct HeapItem {
7      count: usize,
8      pair: (u32, u32),
9  }
10
11  impl Ord for HeapItem {
12      fn cmp(&self, other: &Self) -> Ordering {
13          self.count.cmp(&other.count)
14              .then_with(|| self.pair.cmp(&other.pair))
15      }
16  }
17
18  impl PartialOrd for HeapItem {
19      fn partial_cmp(&self, other: &Self) -> Option<Ordering> {
20          Some(self.cmp(other))
21      }
22  }
23
24  #[pyfunction]
25  fn train_bpe_rust(
26      global_counts: HashMap<Vec<u8>, usize>,
27      target_vocab_size: usize,
28  ) -> PyResult<(Vec<(Vec<u8>, Vec<u8>>), HashMap<usize, Vec<u8>>>) {
29
30      let mut vocab: HashMap<u32, Vec<u8>> = (0..256)
31          .map(|i| (i as u32, vec![i as u8]))
32          .collect();
33      let mut next_id = 256u32;
34      let num_merges = target_vocab_size.saturating_sub(256);
35      let mut merges = Vec::with_capacity(num_merges);
36
37      let mut words: Vec<Vec<u32>> = Vec::new();
38      let mut word_freqs: Vec<usize> = Vec::new();
39
40      for (bytes, freq) in global_counts {
41          words.push(bytes.into_iter().map(|b| b as u32).collect());
42          word_freqs.push(freq);
43      }
44
45      let mut pair_counts: HashMap<(u32, u32), usize> = HashMap::new();
46      let mut pair_to_words: HashMap<(u32, u32), HashSet<usize>> = HashMap::new();
47
48      for (idx, word_ids) in words.iter().enumerate() {
49          let freq = word_freqs[idx];
50          for pair in word_ids.windows(2) {
51              let p = (pair[0], pair[1]);
52              *pair_counts.entry(p).or_insert(0) += freq;
53              pair_to_words.entry(p).or_insert_with(HashSet::new).insert(idx);
54          }

```

```

55     }
56
57     let mut heap: BinaryHeap<HeapItem> = pair_counts.iter()
58         .map(|(&pair, &count)| HeapItem { count, pair })
59         .collect();
60
61     for _ in 0..num_merges {
62         let mut best_pair = None;
63         while let Some(top) = heap.pop() {
64             if let Some(&current_count) = pair_counts.get(&top.pair) {
65                 if current_count == top.count {
66                     best_pair = Some(top.pair);
67                     break;
68                 }
69             }
70         }
71
72         let p = match best_pair {
73             Some(p) => p,
74             None => break,
75         };
76
77         let p1_bytes = vocab.get(&p.0).unwrap().clone();
78         let p2_bytes = vocab.get(&p.1).unwrap().clone();
79         let mut new_token_bytes = p1_bytes;
80         new_token_bytes.extend(p2_bytes);
81
82         merges.push((vocab.get(&p.0).unwrap().clone(), vocab.get(&p.1).unwrap().clone()));
83         vocab.insert(next_id, new_token_bytes);
84
85         let new_id = next_id;
86         next_id += 1;
87
88         if let Some(word_indices) = pair_to_words.remove(&p) {
89             pair_counts.remove(&p);
90
91             for &word_idx in &word_indices {
92                 let freq = word_freqs[word_idx];
93                 let old_ids = &words[word_idx];
94                 let mut new_ids = Vec::new();
95
96                 let mut i = 0;
97                 while i < old_ids.len() {
98                     if i < old_ids.len() - 1 && old_ids[i] == p.0 && old_ids[i+1] == p.1 {
99                         new_ids.push(new_id);
100                         i += 2;
101                     } else {
102                         new_ids.push(old_ids[i]);
103                         i += 1;
104                     }
105                 }
106
107                 for pair in old_ids.windows(2) {
108                     let old_p = (pair[0], pair[1]);
109                     if let Some(count) = pair_counts.get_mut(&old_p) {
110                         *count -= freq;
111                     }
112                 }
113             }

```

```

114         words[word_idx] = new_ids;
115         let updated_ids = &words[word_idx];
116         for pair in updated_ids.windows(2) {
117             let new_p = (pair[0], pair[1]);
118             let count = pair_counts.entry(new_p).or_insert(0);
119             *count += freq;
120             pair_to_words.entry(new_p).or_insert_with(HashSet::new).insert(word_idx);
121             heap.push(HeapItem { count: *count, pair: new_p });
122         }
123     }
124 }
125 }
126
127 let final_vocab = vocab.into_iter().map(|(k, v)| (k as usize, v)).collect();
128 Ok((merges, final_vocab))
129 }
130
131 #[pymodule]
132 fn fast_bpe(m: &Bound<'_, pyo3::types::PyModule>) -> PyResult<()> {
133     m.add_function(wrap_pyfunction!(train_bpe_rust, m)?);
134     Ok(())
135 }

```

A.2 Python BPE Trainer

```

1  import os
2  import time
3  import pickle
4  import regex as re
5  import multiprocessing
6  from typing import BinaryIO
7  from collections import Counter
8
9  import fast_bpe
10
11  # =====
12  # 1. 全局配置与正则定义
13  # =====
14
15  PATTERN = r"('?:[sdmt]|ll|ve|re)| ?\p{L}+| ?\p{N}+| ?[\s\p{L}\p{N}]+\s+(?!S)\s+"
16  PRETOKEN_REGEX = re.compile(PATTERN)
17
18  # =====
19  # 2. 多进程预分词辅助函数 (保持原样, Python 的强项)
20  # =====
21
22  def process_chunk_with_special_tokens(chunk_text: str, special_tokens: list[str], pretoken_regex: re.Pattern) ->
↳ Counter:
23      split_pattern = "|".join(map(re.escape, special_tokens)) if special_tokens else "(?!)"
24      text_segments = re.split(split_pattern, chunk_text)
25      chunk_counts = Counter()
26      for segment in text_segments:
27          if not segment: continue
28          for match in pretoken_regex.finditer(segment):
29              pre_token = match.group()
30              byte_token = pre_token.encode("utf-8")
31              chunk_counts[byte_token] += 1
32      return chunk_counts
33
34  def find_chunk_boundaries(file: BinaryIO, desired_num_chunks: int, split_special_token: bytes) -> list[int]:
35      if not split_special_token:
36          file.seek(0, os.SEEK_END)
37          file_size = file.tell()
38          chunk_size = file_size // desired_num_chunks
39          boundaries = [i * chunk_size for i in range(desired_num_chunks + 1)]
40          boundaries[-1] = file_size
41          return sorted(set(boundaries))
42
43      assert isinstance(split_special_token, bytes)
44      file.seek(0, os.SEEK_END)
45      file_size = file.tell()
46      file.seek(0)
47      chunk_size = file_size // desired_num_chunks
48      chunk_boundaries = [i * chunk_size for i in range(desired_num_chunks + 1)]
49      chunk_boundaries[-1] = file_size
50      mini_chunk_size = 4096
51      for bi in range(1, len(chunk_boundaries) - 1):
52          initial_position = chunk_boundaries[bi]
53          file.seek(initial_position)
54          while True:
55              mini_chunk = file.read(mini_chunk_size)
56              if mini_chunk == b"":

```

```

57         chunk_boundaries[bi] = file_size
58         break
59     found_at = mini_chunk.find(split_special_token)
60     if found_at != -1:
61         chunk_boundaries[bi] = initial_position + found_at
62         break
63     initial_position += mini_chunk_size
64     return sorted(set(chunk_boundaries))
65
66 def worker_task(file_path: str, start: int, end: int, special_tokens: list[str]) -> Counter:
67     with open(file_path, "rb") as f:
68         f.seek(start)
69         chunk_bytes = f.read(end - start)
70         chunk_text = chunk_bytes.decode("utf-8", errors="ignore")
71         return process_chunk_with_special_tokens(chunk_text, special_tokens, PRETOKEN_REGEX)
72
73 def run_parallel_pretokenization(input_path: str, special_tokens: list[str]) -> Counter:
74     num_workers = multiprocessing.cpu_count()
75     num_tasks = num_workers * 4
76     boundary_token = special_tokens[0].encode('utf-8') if special_tokens else b""
77     with open(input_path, "rb") as f:
78         boundaries = find_chunk_boundaries(f, num_tasks, boundary_token)
79     with multiprocessing.Pool(processes=num_workers) as pool:
80         tasks = [(input_path, start, end, special_tokens) for start, end in zip(boundaries[:-1], boundaries[1:])]
81         partial_counts = pool.starmap(worker_task, tasks)
82     global_counts = Counter()
83     for local_count in partial_counts:
84         global_counts.update(local_count)
85     return global_counts
86
87
88 # =====
89 # 3. 核心大作业交付函数: train_bpe
90 # =====
91
92 def train_bpe(input_path: str, vocab_size: int, special_tokens: list[str]) -> tuple[dict[int, bytes],
93 ↪ list[tuple[bytes, bytes]]]:
94     print(" 开始预分词与频率统计 (Python 多进程)...")
95     global_counts = run_parallel_pretokenization(input_path, special_tokens)
96
97     num_merges = vocab_size - 256 - len(special_tokens)
98     if num_merges < 0:
99         raise ValueError("vocab_size is too small to fit base bytes and special tokens.")
100
101     print(f" 将数据移交 Rust 底层进行合并 (目标总词表大小: {vocab_size - len(special_tokens)})...")
102
103     merges, vocab = fast_bpe.train_bpe_rust(global_counts, vocab_size - len(special_tokens))
104
105     print("Rust 计算完成, 添加 Special Tokens...")
106
107     vocab_idx = 256 + len(merges)
108     for st in special_tokens:
109         vocab[vocab_idx] = st.encode("utf-8")
110         vocab_idx += 1
111
112     print("BPE 训练彻底完成!")
113     return vocab, merges
114

```

```

115 # =====
116 # 测试/执行入口
117 # =====
118 if __name__ == "__main__":
119
120     file_path = "/home/inferior/Projects/CS336/data/owt_train.txt"
121     # file_path = "/home/inferior/Projects/CS336/data/TinyStoriesV2-GPT4-train.txt"
122     target_vocab_size = 32000
123     # target_vocab_size = 10000
124     special_tks = ["<|endoftext|>"]
125
126     print("--- 开始 BPE 训练任务 ---")
127     start_time = time.time()
128
129     vocab, merges = train_bpe(
130         input_path=file_path,
131         vocab_size=target_vocab_size,
132         special_tokens=special_tks
133     )
134
135     end_time = time.time()
136     print(f"\n 训练总耗时: {end_time - start_time:.2f} 秒")
137     print(f" 最终合并列表长度: {len(merges)}")
138     print(f" 最终词表大小: {len(vocab)}")
139
140     # -----
141     # 序列化保存 (Serialization)
142     # -----
143     output_dir = "tokenizer_outputs"
144     os.makedirs(output_dir, exist_ok=True)
145
146     vocab_path = os.path.join(output_dir, "vocab.pkl")
147     merges_path = os.path.join(output_dir, "merges.pkl")
148
149     with open(vocab_path, "wb") as f:
150         pickle.dump(vocab, f)
151
152     with open(merges_path, "wb") as f:
153         pickle.dump(merges, f)
154
155     print(f" 序列化完成! 文件已保存至: {output_dir}/")
156
157     # -----
158     # 分析: 寻找最长的 Token (作业要求)
159     # -----
160     longest_token_id = max(vocab, key=lambda k: len(vocab[k]))
161     longest_token_bytes = vocab[longest_token_id]
162
163     print("\n 词表分析:")
164     print(f" 最长 Token 的 ID: {longest_token_id}")
165     print(f" 最长 Token 的字节长度: {len(longest_token_bytes)}")
166     try:
167         print(f" 最长 Token 的文本内容: {longest_token_bytes.decode('utf-8')!r}")
168     except UnicodeDecodeError:
169         print(f" 最长 Token (原始字节): {longest_token_bytes}")
170
171     print("\n最初的 5 次 Merge:")
172     for i in range(min(5, len(merges))):
173         p1, p2 = merges[i]

```

```
174     print(f" {p1} + {p2} -> {p1 + p2}")
```

A.3 Tokenizer

```

1  import pickle
2  import regex as re
3  from typing import Iterable, Iterator
4
5  # =====
6  # 1. 全局配置与正则定义
7  # =====
8  PATTERN = r"\"(?:[sdmt]|ll|ve|re)| ?\p{L}+| ?\p{N}+| ?[\s\p{L}\p{N}]+\s+(?!\\S)\\s+\""
9  PRETOKEN_REGEX = re.compile(PATTERN)
10
11 # =====
12 # 2. Tokenizer 核心类
13 # =====
14 class Tokenizer:
15     def __init__(self, vocab: dict[int, bytes], merges: list[tuple[bytes, bytes]], special_tokens: list[str] |
        ↳ None = None):
16         """
17         初始化 Tokenizer, 建立查找索引并处理特殊 Token。
18         """
19         self.vocab = vocab.copy()
20         self.merges = merges
21         self.special_tokens = special_tokens or []
22
23         # 建立反向查找字典, 用于快速获取 token 对应的 ID
24         self.token_to_id = {v: k for k, v in self.vocab.items()}
25
26         # 将 merges 列表转化为 {pair: rank} 字典, 以 O(1) 的时间复杂度获取合并优先级
27         self.merge_ranks = {pair: i for i, pair in enumerate(self.merges)}
28
29         # 动态处理传入的 special_tokens: 如果不在词表中, 则追加到末尾
30         next_id = max(self.vocab.keys()) + 1 if self.vocab else 0
31         for st in self.special_tokens:
32             st_bytes = st.encode("utf-8")
33             if st_bytes not in self.token_to_id:
34                 self.vocab[next_id] = st_bytes
35                 self.token_to_id[st_bytes] = next_id
36                 next_id += 1
37
38         # 编译用于切分特殊 Token 的正则表达式
39         self.st_pattern = None
40         if self.special_tokens:
41             # 使用捕获组 (...), 这样 re.split 时能保留特殊 token 在列表中
42             sorted_st = sorted(self.special_tokens, key=len, reverse=True)
43             pattern_str = "|".join(map(re.escape, sorted_st))
44             self.st_pattern = re.compile(f"({pattern_str})")
45
46     @classmethod
47     def from_files(cls, vocab_filepath: str, merges_filepath: str, special_tokens: list[str] | None = None):
48         """
49         从序列化的 pickle 文件中加载 vocab 和 merges 并实例化 Tokenizer。
50         """
51         with open(vocab_filepath, "rb") as f:
52             vocab = pickle.load(f)
53         with open(merges_filepath, "rb") as f:
54             merges = pickle.load(f)
55
56         return cls(vocab, merges, special_tokens)

```

```

57
58 def _bpe_encode_chunk(self, chunk_text: str) -> list[int]:
59     """
60     内部辅助函数：对单个正则切割后的纯文本 pre-token 进行 BPE 合并操作。
61     """
62     # 将字符串打散成单字节列表
63     b_list = [bytes([b]) for b in chunk_text.encode("utf-8")]
64
65     # 如果长度为 1，直接返回（基础 256 字节必然在词表中）
66     if len(b_list) <= 1:
67         return [self.token_to_id[b] for b in b_list]
68
69     while True:
70         # 找到当前序列中，在 merges 中优先级最高（rank 最小）的相邻字节对
71         min_rank = float('inf')
72         best_pair = None
73
74         for i in range(len(b_list) - 1):
75             pair = (b_list[i], b_list[i+1])
76             rank = self.merge_ranks.get(pair)
77             if rank is not None and rank < min_rank:
78                 min_rank = rank
79                 best_pair = pair
80
81         # 如果没有任何一对在 merge_ranks 中，说明合并彻底完成
82         if best_pair is None:
83             break
84
85         # 执行最高优先级的一轮合并
86         new_b_list = []
87         i = 0
88         while i < len(b_list):
89             # 命中目标 pair
90             if i < len(b_list) - 1 and b_list[i] == best_pair[0] and b_list[i+1] == best_pair[1]:
91                 new_b_list.append(best_pair[0] + best_pair[1])
92                 i += 2
93             else:
94                 new_b_list.append(b_list[i])
95                 i += 1
96
97         b_list = new_b_list
98
99         # 如果只剩下一个完整的 token，提前退出
100         if len(b_list) == 1:
101             break
102
103         # 拿着最终合并好的字节序列，去词表里换取对应的 IDs
104         return [self.token_to_id[b] for b in b_list]
105
106 def encode(self, text: str) -> list[int]:
107     """
108     将整段输入文本编码为 token IDs。
109     """
110     ids = []
111
112     # 1. 全局截断：优先将文本通过 special_tokens 切开
113     if self.st_pattern:
114         parts = self.st_pattern.split(text)
115     else:

```

```

116         parts = [text]
117
118     for part in parts:
119         if not part:
120             continue
121
122     # 2. 如果这块 part 是一个 special token, 直接查 ID
123     if part in self.special_tokens:
124         ids.append(self.token_to_id[part.encode("utf-8")])
125     else:
126         # 3. 对普通文本进行预分词匹配
127         for match in PRETOKEN_REGEX.finditer(part):
128             chunk_text = match.group()
129             ids.extend(self._bpe_encode_chunk(chunk_text))
130
131     return ids
132
133 def encode_iterable(self, iterable: Iterable[str]) -> Iterator[int]:
134     """
135     懒加载生成器：逐块/逐行读取流数据并输出 IDs, 确保内存开销维持在常数级别。
136     说明：对于标准的文件句柄 (File Handle), 每次 yield 的是一行。
137     换行符属于 BPE 空格家族, 因此按行拆分不会截断合法的词组边界。
138     """
139     for chunk in iterable:
140         yield from self.encode(chunk)
141
142 def decode(self, ids: list[int]) -> str:
143     """
144     将整数 ID 序列还原为字符串。非法字节将被自动替换为 U+FFFD ()。
145     """
146     raw_bytes = b"".join(self.vocab[idx] for idx in ids)
147     return raw_bytes.decode("utf-8", errors="replace")

```

A.4 Adapters

```

1  from __future__ import annotations
2
3  import os
4  import math
5  import collections
6  import json
7  import regex
8  from collections.abc import Iterable
9  from typing import IO, Any, BinaryIO, Dict, List, Tuple
10
11 import numpy.typing as npt
12 import torch
13 from jaxtyping import Bool, Float, Int
14 from torch import Tensor
15 from collections import defaultdict
16 import pickle
17
18 from cs336_basics.test_function.bpe_trainer import train_bpe
19 from cs336_basics.test_function.tokenizer import Tokenizer
20 from cs336_basics.Chap_2.linear import Linear
21 from cs336_basics.Chap_2.embedding import Embedding
22 from cs336_basics.Chap_2.rmsnorm import RMSNorm
23 from cs336_basics.Chap_2.swiglu import SwiGLU
24 from cs336_basics.Chap_2.rope import RotaryPositionalEmbedding
25
26 def run_linear(
27     d_in: int,
28     d_out: int,
29     weights: Float[Tensor, " d_out d_in"],
30     in_features: Float[Tensor, " ... d_in"],
31 ) -> Float[Tensor, " ... d_out"]:
32     """
33     Given the weights of a Linear layer, compute the transformation of a batched input.
34
35     Args:
36         in_dim (int): The size of the input dimension
37         out_dim (int): The size of the output dimension
38         weights (Float[Tensor, "d_out d_in"]): The linear weights to use
39         in_features (Float[Tensor, "... d_in"]): The output tensor to apply the function to
40
41     Returns:
42         Float[Tensor, "... d_out"]: The transformed output of your linear module.
43     """
44
45     model = Linear(
46         in_features=d_in,
47         out_features=d_out,
48         device=in_features.device,
49         dtype=in_features.dtype,
50     )
51
52     model.load_state_dict({"W": weights}, strict=True)
53
54     model.eval()
55     with torch.no_grad():
56         output = model(in_features)
57

```



```

58     return output
59
60
61 def run_embedding(
62     vocab_size: int,
63     d_model: int,
64     weights: Float[Tensor, " vocab_size d_model"],
65     token_ids: Int[Tensor, " ..."],
66 ) -> Float[Tensor, " ... d_model"]:
67     """
68     Given the weights of an Embedding layer, get the embeddings for a batch of token ids.
69
70     Args:
71         vocab_size (int): The number of embeddings in the vocabulary
72         d_model (int): The size of the embedding dimension
73         weights (Float[Tensor, "vocab_size d_model"]): The embedding vectors to fetch from
74         token_ids (Int[Tensor, "..."]): The set of token ids to fetch from the Embedding layer
75
76     Returns:
77         Float[Tensor, "... d_model"]: Batch of embeddings returned by your Embedding layer.
78     """
79
80     model = Embedding(
81         num_embeddings=vocab_size,
82         embedding_dim=d_model,
83         device=weights.device,
84         dtype=weights.dtype
85     )
86
87     model.load_state_dict({"weight": weights}, strict=True)
88
89     model.eval()
90     with torch.no_grad():
91         output = model(token_ids)
92
93     return output
94
95
96 def run_swiglu(
97     d_model: int,
98     d_ff: int,
99     w1_weight: Float[Tensor, " d_ff d_model"],
100     w2_weight: Float[Tensor, " d_model d_ff"],
101     w3_weight: Float[Tensor, " d_ff d_model"],
102     in_features: Float[Tensor, " ... d_model"],
103 ) -> Float[Tensor, " ... d_model"]:
104     """Given the weights of a SwiGLU network, return
105     the output of your implementation with these weights.
106
107     Args:
108         d_model (int): Dimensionality of the feedforward input and output.
109         d_ff (int): Dimensionality of the up-project happening internally to your swiglu.
110         w1_weight (Float[Tensor, "d_ff d_model"]): Stored weights for W1
111         w2_weight (Float[Tensor, "d_model d_ff"]): Stored weights for W2
112         w3_weight (Float[Tensor, "d_ff d_model"]): Stored weights for W3
113         in_features (Float[Tensor, "... d_model"]): Input embeddings to the feed-forward layer.
114
115     Returns:
116         Float[Tensor, "... d_model"]: Output embeddings of the same shape as the input embeddings.

```

```

117     """
118     # Example:
119     # If your state dict keys match, you can use `load_state_dict()`
120     # swiglu.load_state_dict(weights)
121     # You can also manually assign the weights
122     # swiglu.w1.weight.data = w1_weight
123     # swiglu.w2.weight.data = w2_weight
124     # swiglu.w3.weight.data = w3_weight
125     model = SwiGLU(
126         d_model=d_model,
127         d_ff=d_ff,
128         device=in_features.device,
129         dtype=in_features.dtype
130     )
131
132     state_dict = {
133         "w1.W": w1_weight,
134         "w2.W": w2_weight,
135         "w3.W": w3_weight
136     }
137     model.load_state_dict(state_dict, strict=True)
138
139     model.eval()
140     with torch.no_grad():
141         output = model(in_features)
142
143     return output
144
145
146 def run_scaled_dot_product_attention(
147     Q: Float[Tensor, " ... queries d_k"],
148     K: Float[Tensor, " ... keys d_k"],
149     V: Float[Tensor, " ... values d_v"],
150     mask: Bool[Tensor, " ... queries keys"] | None = None,
151 ) -> Float[Tensor, " ... queries d_v"]:
152     """
153     Given key (K), query (Q), and value (V) tensors, return
154     the output of your scaled dot product attention implementation.
155
156     Args:
157         Q (Float[Tensor, " ... queries d_k"]): Query tensor
158         K (Float[Tensor, " ... keys d_k"]): Key tensor
159         V (Float[Tensor, " ... values d_v"]): Values tensor
160         mask (Bool[Tensor, " ... queries keys"] | None): Mask tensor
161
162     Returns:
163         Float[Tensor, " ... queries d_v"]: Output of SDPA
164     """
165     # 提取 d_k 的维度大小, 用于后续的缩放
166     d_k = Q.size(-1)
167
168     # 1. 计算得分矩阵并缩放: (Q @ K^T) / sqrt(d_k)
169     # K.transpose(-2, -1) 的魔法: 只反转最后两个维度 (keys 和 d_k),
170     # 完全无视前面有多少个 batch 维度 (...), 极其优雅安全!
171     # 结果 scores 的形状将会是: (... queries, keys)
172     scores = (Q @ K.transpose(-2, -1)) / math.sqrt(d_k)
173
174     # 2. 应用掩码 (Masking)
175     if mask is not None:
176         # mask 中值为 True 表示“允许看”, False 表示“遮挡不允许看”。

```

```

176         # -mask 取反后, 把所有不允许看的位置强制填充为 -inf
177         scores = scores.masked_fill(~mask, float('-inf'))
178
179     # 3. Softmax 归一化
180     # 沿着最后一个维度 (keys) 将得分转化为概率分布。
181     # 这里你可以调用上一题自己手写的 run_softmax(scores, dim=-1),
182     # 但为了保证在多维 Tensor 上的极致性能, 直接调用内置的 torch.softmax 也是绝佳选择。
183     attention_weights = torch.softmax(scores, dim=-1)
184
185     # 4. 乘以 V (提取信息)
186     # attention_weights 形状: (... queries, keys)
187     # V 形状: (... keys, d_v)
188     # 矩阵乘法后得到最终结果形状: (... queries, d_v)
189     output = attention_weights @ V
190
191     return output
192
193
194 def run_multihead_self_attention(
195     d_model: int,
196     num_heads: int,
197     q_proj_weight: Float[Tensor, "d_k d_in"],
198     k_proj_weight: Float[Tensor, "d_k d_in"],
199     v_proj_weight: Float[Tensor, "d_v d_in"],
200     o_proj_weight: Float[Tensor, "d_model d_v"],
201     in_features: Float[Tensor, "... sequence_length d_in"],
202 ) -> Float[Tensor, "... sequence_length d_out"]:
203     """
204     Given the key, query, and value projection weights of a naive unbatched
205     implementation of multi-head attention, return the output of an optimized batched
206     implementation. This implementation should handle the key, query, and value projections
207     for all heads in a single matrix multiply.
208     This function should not use RoPE.
209     See section 3.2.2 of Vaswani et al., 2017.
210
211     Args:
212         d_model (int): Dimensionality of the feedforward input and output.
213         num_heads (int): Number of heads to use in multi-headed attention.
214         max_seq_len (int): Maximum sequence length to pre-cache if your implementation does that.
215         q_proj_weight (Float[Tensor, "d_k d_in"]): Weights for the Q projection
216         k_proj_weight (Float[Tensor, "d_k d_in"]): Weights for the K projection
217         v_proj_weight (Float[Tensor, "d_k d_in"]): Weights for the V projection
218         o_proj_weight (Float[Tensor, "d_model d_v"]): Weights for the output projection
219         in_features (Float[Tensor, "... sequence_length d_in"]): Tensor to run your implementation on.
220
221     Returns:
222         Float[Tensor, "... sequence_length d_out"]: Tensor with the output of running your optimized, batched
223         ↪ multi-headed attention
224         implementation with the given QKV projection weights and input features.
225     """
226     raise NotImplementedError
227
228 def run_multihead_self_attention_with_rope(
229     d_model: int,
230     num_heads: int,
231     max_seq_len: int,
232     theta: float,
233     q_proj_weight: Float[Tensor, "d_k d_in"],

```

```

234     k_proj_weight: Float[Tensor, " d_k d_in"],
235     v_proj_weight: Float[Tensor, " d_v d_in"],
236     o_proj_weight: Float[Tensor, " d_model d_v"],
237     in_features: Float[Tensor, " ... sequence_length d_in"],
238     token_positions: Int[Tensor, " ... sequence_length"] | None = None,
239 ) -> Float[Tensor, " ... sequence_length d_out"]:
240     """
241     Given the key, query, and value projection weights of a naive unbatched
242     implementation of multi-head attention, return the output of an optimized batched
243     implementation. This implementation should handle the key, query, and value projections
244     for all heads in a single matrix multiply.
245     This version of MHA should include RoPE.
246     In this case, the RoPE embedding dimension must be the head embedding dimension (d_model // num_heads).
247     See section 3.2.2 of Vaswani et al., 2017.
248
249     Args:
250         d_model (int): Dimensionality of the feedforward input and output.
251         num_heads (int): Number of heads to use in multi-headed attention.
252         max_seq_len (int): Maximum sequence length to pre-cache if your implementation does that.
253         theta (float): RoPE parameter.
254         q_proj_weight (Float[Tensor, "d_k d_in"]): Weights for the Q projection
255         k_proj_weight (Float[Tensor, "d_k d_in"]): Weights for the K projection
256         v_proj_weight (Float[Tensor, "d_k d_in"]): Weights for the V projection
257         o_proj_weight (Float[Tensor, "d_model d_v"]): Weights for the output projection
258         in_features (Float[Tensor, "... sequence_length d_in"]): Tensor to run your implementation on.
259         token_positions (Int[Tensor, " ... sequence_length"] | None): Optional tensor with the positions of the
260             ↪ tokens
261
262     Returns:
263         Float[Tensor, " ... sequence_length d_out"]: Tensor with the output of running your optimized, batched
264             ↪ multi-headed attention
265         implementation with the given QKV projection weights and input features.
266     """
267     raise NotImplementedError
268
269 def run_rope(
270     d_k: int,
271     theta: float,
272     max_seq_len: int,
273     in_query_or_key: Float[Tensor, " ... sequence_length d_k"],
274     token_positions: Int[Tensor, " ... sequence_length"],
275 ) -> Float[Tensor, " ... sequence_length d_k"]:
276     """
277     Run RoPE for a given input tensor.
278
279     Args:
280         d_k (int): Embedding dimension size for the query or key tensor.
281         theta (float): RoPE parameter.
282         max_seq_len (int): Maximum sequence length to pre-cache if your implementation does that.
283         in_query_or_key (Float[Tensor, "... sequence_length d_k"]): Input tensor to run RoPE on.
284         token_positions (Int[Tensor, "... sequence_length"]): Tensor of shape (batch_size, sequence_length) with
285             ↪ the token positions
286
287     Returns:
288         Float[Tensor, " ... sequence_length d_k"]: Tensor with RoPEd input.
289     """
290     # 1. 实例化 RoPE 模块
291     model = RotaryPositionalEmbedding(
292         theta=theta,

```

```

290         d_k=d_k,
291         max_seq_len=max_seq_len,
292         device=in_query_or_key.device
293     )
294
295
296     # 2. 评估模式并切断梯度, 直接计算
297     model.eval()
298     with torch.no_grad():
299         # 注意要把 token_positions 传进去!
300         output = model(in_query_or_key, token_positions)
301
302     return output
303
304
305 def run_transformer_block(
306     d_model: int,
307     num_heads: int,
308     d_ff: int,
309     max_seq_len: int,
310     theta: float,
311     weights: dict[str, Tensor],
312     in_features: Float[Tensor, " batch sequence_length d_model"],
313 ) -> Float[Tensor, " batch sequence_length d_model"]:
314     """
315     Given the weights of a pre-norm Transformer block and input features,
316     return the output of running the Transformer block on the input features.
317
318     This function should use RoPE.
319     Depending on your implementation, you may simply need to pass the relevant args
320     to your TransformerBlock constructor, or you may need to initialize your own RoPE
321     class and pass that instead.
322
323     Args:
324         d_model (int): The dimensionality of the Transformer block input.
325         num_heads (int): Number of heads to use in multi-headed attention. `d_model` must be
326             evenly divisible by `num_heads`.
327         d_ff (int): Dimensionality of the feed-forward inner layer.
328         max_seq_len (int): Maximum sequence length to pre-cache if your implementation does that.
329         theta (float): RoPE parameter.
330         weights (dict[str, Tensor]):
331             State dict of our reference implementation.
332             The keys of this dictionary are:
333             - `attn.q_proj.weight`
334                 The query projections for all `num_heads` attention heads.
335                 Shape is (d_model, d_model).
336                 The rows are ordered by matrices of shape (num_heads, d_k),
337                 so `attn.q_proj.weight == torch.cat([q_heads.0.weight, ..., q_heads.N.weight], dim=0)`.
338             - `attn.k_proj.weight`
339                 The key projections for all `num_heads` attention heads.
340                 Shape is (d_model, d_model).
341                 The rows are ordered by matrices of shape (num_heads, d_k),
342                 so `attn.k_proj.weight == torch.cat([k_heads.0.weight, ..., k_heads.N.weight], dim=0)`.
343             - `attn.v_proj.weight`
344                 The value projections for all `num_heads` attention heads.
345                 Shape is (d_model, d_model).
346                 The rows are ordered by matrices of shape (num_heads, d_v),
347                 so `attn.v_proj.weight == torch.cat([v_heads.0.weight, ..., v_heads.N.weight], dim=0)`.
348             - `attn.output_proj.weight`

```

```

349         Weight of the multi-head self-attention output projection
350         Shape is (d_model, d_model).
351     - `ln1.weight`
352         Weights of affine transform for the first RMSNorm
353         applied in the transformer block.
354         Shape is (d_model,).
355     - `ffn.w1.weight`
356         Weight of the first linear transformation in the FFN.
357         Shape is (d_model, d_ff).
358     - `ffn.w2.weight`
359         Weight of the second linear transformation in the FFN.
360         Shape is (d_ff, d_model).
361     - `ffn.w3.weight`
362         Weight of the third linear transformation in the FFN.
363         Shape is (d_model, d_ff).
364     - `ln2.weight`
365         Weights of affine transform for the second RMSNorm
366         applied in the transformer block.
367         Shape is (d_model,).
368     in_features (Float[Tensor, "batch sequence_length d_model"]):
369     Tensor to run your implementation on.
370
371     Returns:
372     Float[Tensor, "batch sequence_length d_model"] Tensor with the output of
373     running the Transformer block on the input features while using RoPE.
374     """
375     raise NotImplementedError
376
377
378 def run_transformer_lm(
379     vocab_size: int,
380     context_length: int,
381     d_model: int,
382     num_layers: int,
383     num_heads: int,
384     d_ff: int,
385     rope_theta: float,
386     weights: dict[str, Tensor],
387     in_indices: Int[Tensor, " batch_size sequence_length"],
388 ) -> Float[Tensor, " batch_size sequence_length vocab_size"]:
389     r"""Given the weights of a Transformer language model and input indices,
390     return the output of running a forward pass on the input indices.
391
392     This function should use RoPE.
393
394     Args:
395     vocab_size (int): The number of unique items in the output vocabulary to be predicted.
396     context_length (int): The maximum number of tokens to process at once.
397     d_model (int): The dimensionality of the model embeddings and sublayer outputs.
398     num_layers (int): The number of Transformer layers to use.
399     num_heads (int): Number of heads to use in multi-headed attention. `d_model` must be
400     evenly divisible by `num_heads`.
401     d_ff (int): Dimensionality of the feed-forward inner layer (section 3.3).
402     rope_theta (float): The RoPE  $\Theta$  parameter.
403     weights (dict[str, Tensor]):
404     State dict of our reference implementation. {num_layers} refers to an
405     integer between `0` and `num_layers - 1` (the layer index).
406     The keys of this dictionary are:
407     - `token_embeddings.weight`

```

```

408         Token embedding matrix. Shape is (vocab_size, d_model).
409     - `layers.{num_layers}.attn.q_proj.weight`
410         The query projections for all `num_heads` attention heads.
411         Shape is (num_heads * (d_model / num_heads), d_model).
412         The rows are ordered by matrices of shape (num_heads, d_k),
413         so `attn.q_proj.weight == torch.cat([q_heads.0.weight, ..., q_heads.N.weight], dim=0)`.
414     - `layers.{num_layers}.attn.k_proj.weight`
415         The key projections for all `num_heads` attention heads.
416         Shape is (num_heads * (d_model / num_heads), d_model).
417         The rows are ordered by matrices of shape (num_heads, d_k),
418         so `attn.k_proj.weight == torch.cat([k_heads.0.weight, ..., k_heads.N.weight], dim=0)`.
419     - `layers.{num_layers}.attn.v_proj.weight`
420         The value projections for all `num_heads` attention heads.
421         Shape is (num_heads * (d_model / num_heads), d_model).
422         The rows are ordered by matrices of shape (num_heads, d_v),
423         so `attn.v_proj.weight == torch.cat([v_heads.0.weight, ..., v_heads.N.weight], dim=0)`.
424     - `layers.{num_layers}.attn.output_proj.weight`
425         Weight of the multi-head self-attention output projection
426         Shape is ((d_model / num_heads) * num_heads, d_model).
427     - `layers.{num_layers}.ln1.weight`
428         Weights of affine transform for the first RMSNorm
429         applied in the transformer block.
430         Shape is (d_model,).
431     - `layers.{num_layers}.ffn.w1.weight`
432         Weight of the first linear transformation in the FFN.
433         Shape is (d_model, d_ff).
434     - `layers.{num_layers}.ffn.w2.weight`
435         Weight of the second linear transformation in the FFN.
436         Shape is (d_ff, d_model).
437     - `layers.{num_layers}.ffn.w3.weight`
438         Weight of the third linear transformation in the FFN.
439         Shape is (d_model, d_ff).
440     - `layers.{num_layers}.ln2.weight`
441         Weights of affine transform for the second RMSNorm
442         applied in the transformer block.
443         Shape is (d_model,).
444     - `ln_final.weight`
445         Weights of affine transform for RMSNorm applied to the output of the final transformer block.
446         Shape is (d_model, ).
447     - `lm_head.weight`
448         Weights of the language model output embedding.
449         Shape is (vocab_size, d_model).
450     in_indices (Int[Tensor, "batch_size sequence_length"]) Tensor with input indices to run the language model
451     ↪ on. Shape is (batch_size, sequence_length), where
452         `sequence_length` is at most `context_length`.
453
454     Returns:
455         Float[Tensor, "batch_size sequence_length vocab_size"]: Tensor with the predicted unnormalized
456         next-word distribution for each token.
457
458     """
459     raise NotImplementedError
460
461 def run_rmsnorm(
462     d_model: int,
463     eps: float,
464     weights: Float[Tensor, " d_model"],
465     in_features: Float[Tensor, " ... d_model"],
466 ) -> Float[Tensor, " ... d_model"]:

```

```

466     """Given the weights of a RMSNorm affine transform,
467 return the output of running RMSNorm on the input features.
468
469     Args:
470         d_model (int): The dimensionality of the RMSNorm input.
471         eps: (float): A value added to the denominator for numerical stability.
472         weights (Float[Tensor, "d_model"]): RMSNorm weights.
473         in_features (Float[Tensor, "... d_model"]): Input features to run RMSNorm on. Can have arbitrary leading
474 dimensions.
475
476     Returns:
477         Float[Tensor, "... d_model"]: Tensor of with the same shape as `in_features` with the output of running
478         RMSNorm of the `in_features`.
479     """
480     model = RMSNorm(
481         d_model=d_model,
482         eps=eps,
483         device=weights.device,
484         dtype=weights.dtype
485     )
486
487     model.load_state_dict({"weight": weights}, strict=True)
488
489     model.eval()
490     with torch.no_grad():
491         output = model(in_features)
492
493     return output
494
495
496 def run_silu(in_features: Float[Tensor, "..."]) -> Float[Tensor, "..."]:
497     """Given a tensor of inputs, return the output of applying SiLU
498 to each element.
499
500     Args:
501         in_features (Float[Tensor, "..."]): Input features to run SiLU on. Shape is arbitrary.
502
503     Returns:
504         Float[Tensor, "..."]: of with the same shape as `in_features` with the output of applying
505         SiLU to each element.
506     """
507     raise NotImplementedError
508
509
510 def run_get_batch(
511     dataset: npt.NDArray, batch_size: int, context_length: int, device: str
512 ) -> tuple[torch.Tensor, torch.Tensor]:
513     """
514 Given a dataset (a 1D numpy array of integers) and a desired batch size and
515 context length, sample language modeling input sequences and their corresponding
516 labels from the dataset.
517
518     Args:
519         dataset (np.array): 1D numpy array of integer token IDs in the dataset.
520         batch_size (int): Desired batch size to sample.
521         context_length (int): Desired context length of each sampled example.
522         device (str): PyTorch device string (e.g., 'cpu' or 'cuda:0') indicating the device
523 to place the sampled input sequences and labels on.
524

```



```

525     Returns:
526         Tuple of torch.LongTensors of shape (batch_size, context_length). The first tuple item
527         is the sampled input sequences, and the second tuple item is the corresponding
528         language modeling labels.
529     """
530     raise NotImplementedError
531
532
533 def run_softmax(in_features: Float[Tensor, "..."], dim: int) -> Float[Tensor, "..."]:
534     """
535     Given a tensor of inputs, return the output of softmaxing the given `dim`
536     of the input.
537
538     Args:
539         in_features (Float[Tensor, "..."]): Input features to softmax. Shape is arbitrary.
540         dim (int): Dimension of the `in_features` to apply softmax to.
541
542     Returns:
543         Float[Tensor, "..."]: Tensor of with the same shape as `in_features` with the output of
544         softmax normalizing the specified `dim`.
545     """
546     # 1. 寻找指定维度上的最大值
547     # keepdim=True 是灵魂! 它能保留维度形状 (比如把形状从 [3, 4] 变成 [3, 1]),
548     # 这样后续减法时才能正确触发广播机制 (Broadcasting)。
549     # 注意: torch.max 会返回一个元组 (values, indices), 我们只需要取第 0 项的 values。
550     max_vals = in_features.max(dim=dim, keepdim=True)[0]
551
552     # 2. Max Trick: 平移输入值, 防止指数爆炸
553     shifted_logits = in_features - max_vals
554
555     # 3. 计算指数 (现在所有的输入都 <= 0, 所以 exp 的结果都在 0~1 之间, 绝对安全)
556     exps = torch.exp(shifted_logits)
557
558     # 4. 沿着指定维度求和, 作为分母
559     sum_exps = exps.sum(dim=dim, keepdim=True)
560
561     # 5. 归一化, 得到最终的概率分布
562     probabilities = exps / sum_exps
563
564     return probabilities
565
566
567 def run_cross_entropy(
568     inputs: Float[Tensor, "batch_size vocab_size"], targets: Int[Tensor, "batch_size"]
569 ) -> Float[Tensor, ""]:
570     """Given a tensor of inputs and targets, compute the average cross-entropy
571     loss across examples.
572
573     Args:
574         inputs (Float[Tensor, "batch_size vocab_size"]): inputs[i][j] is the
575         unnormalized logit of jth class for the ith example.
576         targets (Int[Tensor, "batch_size"]): Tensor of shape (batch_size,) with the index of the correct class.
577         Each value must be between 0 and `num_classes - 1`.
578
579     Returns:
580         Float[Tensor, ""]: The average cross-entropy loss across examples.
581     """
582     raise NotImplementedError
583

```

```

584
585 def run_gradient_clipping(parameters: Iterable[torch.nn.Parameter], max_l2_norm: float) -> None:
586     """Given a set of parameters, clip their combined gradients to have l2 norm at most max_l2_norm.
587
588     Args:
589         parameters (Iterable[torch.nn.Parameter]): collection of trainable parameters.
590         max_l2_norm (float): a positive value containing the maximum l2-norm.
591
592     The gradients of the parameters (parameter.grad) should be modified in-place.
593     """
594     raise NotImplementedError
595
596
597 def get_adamw_cls() -> Any:
598     """
599     Returns a torch.optim.Optimizer that implements AdamW.
600     """
601     raise NotImplementedError
602
603
604 def run_get_lr_cosine_schedule(
605     it: int,
606     max_learning_rate: float,
607     min_learning_rate: float,
608     warmup_iters: int,
609     cosine_cycle_iters: int,
610 ):
611     """
612     Given the parameters of a cosine learning rate decay schedule (with linear
613     warmup) and an iteration number, return the learning rate at the given
614     iteration under the specified schedule.
615
616     Args:
617         it (int): Iteration number to get learning rate for.
618         max_learning_rate (float): alpha_max, the maximum learning rate for
619             cosine learning rate schedule (with warmup).
620         min_learning_rate (float): alpha_min, the minimum / final learning rate for
621             the cosine learning rate schedule (with warmup).
622         warmup_iters (int): T_w, the number of iterations to linearly warm-up
623             the learning rate.
624         cosine_cycle_iters (int): T_c, the number of cosine annealing iterations.
625
626     Returns:
627         Learning rate at the given iteration under the specified schedule.
628     """
629     raise NotImplementedError
630
631
632 def run_save_checkpoint(
633     model: torch.nn.Module,
634     optimizer: torch.optim.Optimizer,
635     iteration: int,
636     out: str | os.PathLike | BinaryIO | IO[bytes],
637 ):
638     """
639     Given a model, optimizer, and an iteration number, serialize them to disk.
640
641     Args:
642         model (torch.nn.Module): Serialize the state of this model.

```

```

643         optimizer (torch.optim.Optimizer): Serialize the state of this optimizer.
644         iteration (int): Serialize this value, which represents the number of training iterations
645             we've completed.
646         out (str | os.PathLike | BinaryIO | IO[bytes]): Path or file-like object to serialize the model,
647             ↪ optimizer, and iteration to.
648     """
649     raise NotImplementedError
650
651 def run_load_checkpoint(
652     src: str | os.PathLike | BinaryIO | IO[bytes],
653     model: torch.nn.Module,
654     optimizer: torch.optim.Optimizer,
655 ) -> int:
656     """
657     Given a serialized checkpoint (path or file-like object), restore the
658     serialized state to the given model and optimizer.
659     Return the number of iterations that we previously serialized in
660     the checkpoint.
661
662     Args:
663         src (str | os.PathLike | BinaryIO | IO[bytes]): Path or file-like object to serialized checkpoint.
664         model (torch.nn.Module): Restore the state of this model.
665         optimizer (torch.optim.Optimizer): Restore the state of this optimizer.
666
667     Returns:
668         int: the previously-serialized number of iterations.
669     """
670     raise NotImplementedError
671
672 def get_tokenizer(
673     vocab: dict[int, bytes],
674     merges: list[tuple[bytes, bytes]],
675     special_tokens: list[str] | None = None,
676 ) -> Any:
677     """Given a vocabulary, a list of merges, and a list of special tokens,
678     return a BPE tokenizer that uses the provided vocab, merges, and special tokens.
679
680     Args:
681         vocab (dict[int, bytes]): The tokenizer vocabulary, a mapping from int (token ID in the vocabulary)
682             to bytes (token bytes)
683         merges (list[tuple[bytes, bytes]]): BPE merges. Each list item is a tuple of bytes (<token1>, <token2>),
684             representing that <token1> was merged with <token2>.
685             Merges are ordered by order of creation.
686         special_tokens (list[str] | None): A list of string special tokens for the tokenizer. These strings will
687             ↪ never
688             be split into multiple tokens, and will always be kept as a single token.
689
690     Returns:
691         A BPE tokenizer that uses the provided vocab, merges, and special tokens.
692     """
693     return Tokenizer(vocab=vocab, merges=merges, special_tokens=special_tokens)
694
695 def run_train_bpe(
696     input_path: str,
697     vocab_size: int,
698     special_tokens: list[str]
699 ) -> tuple[dict[int, bytes], list[tuple[bytes, bytes]]]:

```

```
700     """
701     Adapter function for training the BPE tokenizer.
702     This routes the inputs from the test suite to our optimized implementation.
703     """
704
705     # 直接调用核心逻辑并返回结果
706     vocab, merges = train_bpe(
707         input_path=input_path,
708         vocab_size=vocab_size,
709         special_tokens=special_tokens
710     )
711
712     return vocab, merges
```
